



32. 스마트 포인터와 메모리 관리

● C/C++ 강의 (240 시간 8시간*30일)

32. 스마트 포인터와 메모리 관리

32-1 스마트 포인터 개요

32-1-1 스마트 포인터란?

32-1-2 왜 스마트 포인터가 필요한가?

32-1-3 `std::unique_ptr` vs `std::shared_ptr`

32-1-4 실습: `std::unique_ptr` 로 동적 객체 관리

32-1-5 정리

32-2 `std::unique_ptr` 심화

32-2-1 소유권 이전 (`std::move`)

32-2-2 사용자 정의 Deleter

32-2-3 실습: `unique_ptr` 를 활용한 자원 관리 (파일 핸들)

32-2-4 핵심 요약

32-3 `std::shared_ptr`와 `std::weak_ptr`

32-3-1 참조 카운팅과 순환 참조 문제

순환 참조 문제

32-3-2 `weak_ptr` 로 순환 참조 해결

32-3-3 실습: `shared_ptr` 로 객체 그래프 관리

32-3-4 핵심 요약

32-4 스마트 포인터 모범 사례

32-4-1 `make_unique` , `make_shared` 사용

32-4-2 스마트 포인터와 STL 컨테이너 결합

32-4-3 실습: `shared_ptr` 를 사용한 트리 구조 구현

32-4-4 핵심 요약

32-5 이동 의미론과 Rvalue 참조

32-5-1 이동 의미론과 Rvalue 참조

32-5-2 `std::move` , `std::forward`

32-5-3 복사 vs 이동 성능 비교

32-5-4 실습: 이동 생성자를 가진 클래스 구현

32-5-5 핵심 요약

32-6 템플릿과 STL

32-6-1 템플릿 함수와 클래스 기본

32-6-2 `std::variant` , `std::optional` , `std::any`

`std::variant`

`std::optional`

`std::any`

32-6-3 실습: `optional` 과 `variant` 를 활용한 안전한 데이터 처리

32-6-4 핵심 요약

32-7 멀티스레딩과 동시성

32-7-1 `std::thread` , `std::mutex` , `std::atomic`

32-7-2 `std::async` 와 `std::future`

32-7-3 실습: 멀티스레드 기반 작업 큐 구현

32-7-4 핵심 요약

32-8 최종 프로젝트

32-8-1 주제: STL과 스마트 포인터를 활용한 간단한 데이터베이스 엔진

32-8-2 요구사항

32-8-3 실습: 팀별 구현 및 코드 리뷰

32-8-4 핵심 요약

완성 예제 코드: 멀티스레드 안전 데이터베이스 클래스

32. 스마트 포인터와 메모리 관리

32-1 스마트 포인터 개요

32-1-1 스마트 포인터란?

- 스마트 포인터(Smart Pointer)는 C++에서 메모리 관리를 자동화하기 위해 사용되는 클래스 템플릿입니다.
- 일반적인 포인터와 달리, 객체의 소멸 시점에 자원을 자동으로 해제하여 **메모리 누수 (memory leak)**를 방지합니다.
- 대표적으로 `std::unique_ptr` , `std::shared_ptr` , `std::weak_ptr` 가 있습니다.

32-1-2 왜 스마트 포인터가 필요한가?

RAII(Resource Acquisition Is Initialization)

- 자원 획득을 객체의 생성과 동시에 하고, 해제를 객체의 소멸과 함께하는 C++의 핵심 개념입니다.
- 스마트 포인터는 RAII 개념을 구현하여 예외 상황에서도 안전하게 메모리를 해제합니다.

메모리 누수 방지

- `new` / `delete` 를 수동으로 관리하면 실수로 `delete` 를 빼먹는 일이 잦습니다.
- 특히 예외가 발생하거나 함수에서 조기 리턴되는 경우, 자원 해제가 누락될 수 있습니다.
- 스마트 포인터는 범위를 벗어날 때 자동으로 자원을 해제하므로 안전합니다.

32-1-3 `std::unique_ptr` vs `std::shared_ptr`

특징	<code>std::unique_ptr</code>	<code>std::shared_ptr</code>
소유권	단독 소유 (unique)	참조 카운트 기반 공유 (shared)
복사 가능 여부	불가 (move만 가능)	가능 (카운트 증가)
메모리 해제	객체가 scope 벗어나면 자동 해제	마지막 참조자 소멸 시 해제
오버헤드	없음	참조 카운트 관리 비용 존재
용도	단일 객체 관리	복수 객체 간 공유 필요 시

32-1-4 실습: `std::unique_ptr` 로 동적 객체 관리

```
#include <iostream>
#include <memory>

class MyClass {
public:
    MyClass() { std::cout << "MyClass 생성\\n"; }
    ~MyClass() { std::cout << "MyClass 소멸\\n"; }
    void sayHello() { std::cout << "Hello!\\n"; }
};

int main() {
    std::unique_ptr<MyClass> ptr = std::make_unique<MyClass>();
    ptr->sayHello();
    // ptr은 main 함수 종료 시 자동으로 delete 됨
    return 0;
}
```

실습 요점

- `std::make_unique` 를 통해 객체를 생성하면 예외 안전성이 보장됩니다.
- `unique_ptr` 는 복사할 수 없고, 소유권 이전은 `std::move` 를 통해 수행합니다.

32-1-5 정리

- 스마트 포인터는 C++의 안전하고 효율적인 메모리 관리를 위한 핵심 도구입니다.
- `unique_ptr` 는 소유권이 하나뿐인 자원을 다룰 때 유용하며,
- `shared_ptr` 는 여러 객체가 자원을 공유할 때 적합합니다.
- 수업이나 프로젝트에서 가능한 `new / delete` 대신 **스마트 포인터 사용**을 권장합니다.

참조 사이트

- [Smart pointers \(Modern C++\) - Learn Microsoft](#)
- [Differences between unique_ptr and shared_ptr - Stack Overflow](#)
- [C++ Beyond the Syllabus #4: RAII & Smart Pointers | by Jared Miller](#)
- [Smart pointer - Wikipedia](#)
- [Resource acquisition is initialization - Wikipedia](#)

32-2 std::unique_ptr 심화

32-2-1 소유권 이전 (`std::move`)

- `unique_ptr` 는 복사가 불가능하지만, **소유권을 이동(Move)** 하는 방식으로 다른 포인터로 넘길 수 있습니다.
- 이동 후 원래 포인터는 `nullptr` 로 초기화되며, 더 이상 자원을 소유하지 않습니다.
- 예외 안전성을 확보할 수 있으며, 자원의 단일 소유권을 명확히 표현할 수 있습니다.

```
#include <iostream>
#include <memory>

int main() {
    std::unique_ptr<int> p1 = std::make_unique<int>(100);
    std::unique_ptr<int> p2 = std::move(p1); // 소유권 이동

    if (!p1) std::cout << "p1은 nullptr입니다.\n";
}
```

```
std::cout << "p2가 소유한 값: " << *p2 << "\n";
return 0;
}
```

32-2-2 사용자 정의 Deleter

- `unique_ptr` 는 기본적으로 `delete` 를 사용하지만, **사용자 정의 Deleter**를 지정할 수 있습니다.
- 파일 핸들, 소켓, 커스텀 리소스 해제 등을 처리할 수 있으며, `std::function` 또는 함수 객체 (Functor) 형태로 정의됩니다.

```
#include <memory>
#include <cstdio>

struct FileCloser {
    void operator()(FILE* fp) const {
        if (fp) std::fclose(fp);
    }
};

int main() {
    std::unique_ptr<FILE, FileCloser> filePtr(fopen("test.txt", "r"));
    // 파일 사용
    return 0; // filePtr이 소멸되면서 fclose 자동 호출
}
```

32-2-3 실습: `unique_ptr` 를 활용한 자원 관리 (파일 핸들)

```
#include <iostream>
#include <memory>
#include <cstdio>

struct FileDeleter {
    void operator()(FILE* f) const {
        if (f) {
            std::cout << "파일 닫기\n";
            fclose(f);
        }
    }
};
```

```

    }
}
};

int main() {
    std::unique_ptr<FILE, FileDeleter> fp(fopen("sample.txt", "w"));
    if (fp) {
        fputs("Hello, file!\n", fp.get());
    }
    return 0;
}

```

32-2-4 핵심 요약

- `std::unique_ptr` 는 이동만 가능하므로 소유권의 단일성과 명확성을 보장합니다.
- 사용자 정의 Deleter를 사용하면 다양한 리소스에 대한 자동 해제를 구현할 수 있습니다.
- 실습에서는 파일 핸들 관리에 활용되어, C 스타일 리소스를 안전하게 다루는 방법을 익힐 수 있습니다.

참조 사이트

- [Smart pointers \(Modern C++\) - Learn Microsoft](#)
- [C++ unique_ptr explained](#)
- [Differences between unique_ptr and shared_ptr - Stack Overflow](#)

32-3 std::shared_ptr와 std::weak_ptr

32-3-1 참조 카운팅과 순환 참조 문제

- `std::shared_ptr` 는 참조 카운트를 기반으로 여러 개의 포인터가 같은 객체를 소유할 수 있도록 합니다.
- 내부적으로 참조 카운트가 0이 될 때 객체가 자동으로 해제됩니다.

```

#include <iostream>
#include <memory>

int main() {
    std::shared_ptr<int> a = std::make_shared<int>(10);
    std::shared_ptr<int> b = a; // 참조 카운트 증가
    std::cout << "a use_count: " << a.use_count() << "\n"; // 2
    std::cout << "b use_count: " << b.use_count() << "\n"; // 2
    return 0;
}

```

순환 참조 문제

- 두 객체가 서로를 `shared_ptr` 로 참조할 경우, 참조 카운트가 0이 되지 않아 **메모리 누수**가 발생합니다.

```

#include <memory>

struct B;

struct A {
    std::shared_ptr<B> b_ptr;
};

struct B {
    std::shared_ptr<A> a_ptr;
};

int main() {
    auto a = std::make_shared<A>();
    auto b = std::make_shared<B>();
    a->b_ptr = b;
    b->a_ptr = a; // 순환 참조 발생
    return 0;
} // 메모리 누수 발생

```

32-3-2 `weak_ptr` 로 순환 참조 해결

- `std::weak_ptr` 는 참조 카운트에 영향을 주지 않으며, `shared_ptr` 로부터 객체를 참조만 합니다.
- 순환 참조를 방지할 때 주로 사용되며, `.lock()` 메서드를 통해 `shared_ptr` 로 승격할 수 있습니다.

```
#include <memory>
#include <iostream>

struct B;

struct A {
    std::shared_ptr<B> b_ptr;
    ~A() { std::cout << "A 소멸\n"; }
};

struct B {
    std::weak_ptr<A> a_ptr; // weak_ptr로 순환 참조 해결
    ~B() { std::cout << "B 소멸\n"; }
};

int main() {
    auto a = std::make_shared<A>();
    auto b = std::make_shared<B>();
    a->b_ptr = b;
    b->a_ptr = a;
    return 0;
} // A와 B 모두 소멸됨
```

32-3-3 실습: `shared_ptr` 로 객체 그래프 관리

```
#include <iostream>
#include <memory>
#include <vector>

struct Node {
    int value;
    std::vector<std::shared_ptr<Node>> children;
```



```

Node(int v) : value(v) { std::cout << "Node(" << v << ") 생성\n"; }
~Node() { std::cout << "Node(" << value << ") 소멸\n"; }
};

int main() {
    auto root = std::make_shared<Node>(1);
    root->children.push_back(std::make_shared<Node>(2));
    root->children.push_back(std::make_shared<Node>(3));
    return 0;
}

```

32-3-4 핵심 요약

- `shared_ptr` 는 여러 개체가 동일한 리소스를 소유할 수 있도록 합니다.
- 순환 참조는 객체가 서로를 공유할 때 해제되지 않는 문제를 유발하므로 `weak_ptr` 로 해결해야 합니다.
- 실습 예제를 통해 객체 그래프를 안전하게 관리하는 방법을 익힐 수 있습니다.

참조 사이트

- [shared_ptr - cppreference](#)
- [weak_ptr - cppreference](#)
- [Smart pointers in C++ - Microsoft Docs](#)

32-4 스마트 포인터 모범 사례

32-4-1 `make_unique` , `make_shared` 사용

- `std::make_unique` 와 `std::make_shared` 는 스마트 포인터를 생성할 때 권장되는 함수입니다.
- 생성자 인자를 직접 전달하며, 메모리 할당과 초기화를 안전하게 수행합니다.
- 코드가 간결해지고 예외 안전성을 제공합니다.

```

auto uptr = std::make_unique<int>(10);
auto sptr = std::make_shared<std::string>("Hello");

```

! new 연산자 대신 make 계열 함수를 사용하여 예외 안전성과 가독성 향상

32-4-2 스마트 포인터와 STL 컨테이너 결합

- 스마트 포인터는 STL 컨테이너 (`std::vector` , `std::map` 등)와 함께 사용할 수 있습니다.
- 컨테이너는 객체의 복사나 이동이 자주 발생하므로, 소유권 관리에 유리한 `shared_ptr` 가 자주 쓰입니다.

```
#include <memory>
#include <vector>
#include <iostream>

struct Data {
    int value;
    Data(int v) : value(v) {}
};

int main() {
    std::vector<std::shared_ptr<Data> > dataList;
    dataList.push_back(std::make_shared<Data>(1));
    dataList.push_back(std::make_shared<Data>(2));

    for (const auto& d : dataList) {
        std::cout << d->value << "\n";
    }
    return 0;
}
```

32-4-3 실습: `shared_ptr` 를 사용한 트리 구조 구현

```
#include <iostream>
#include <memory>
#include <vector>

struct TreeNode {
    int value;
    std::vector<std::shared_ptr<TreeNode> > children;
};
```

```

    TreeNode(int val) : value(val) {}
};

void printTree(const std::shared_ptr<TreeNode>& node, int depth = 0) {
    if (!node) return;
    std::cout << std::string(depth * 2, ' ') << node->value << "\n";
    for (const auto& child : node->children) {
        printTree(child, depth + 1);
    }
}

int main() {
    auto root = std::make_shared<TreeNode>(1);
    root->children.push_back(std::make_shared<TreeNode>(2));
    root->children.push_back(std::make_shared<TreeNode>(3));
    root->children[0]->children.push_back(std::make_shared<TreeNode>(4));

    printTree(root);
    return 0;
}

```

32-4-4 핵심 요약

- `make_unique` 와 `make_shared` 는 스마트 포인터 생성 시 기본적으로 사용할 것을 권장합니다.
- STL 컨테이너와 함께 사용할 경우, `shared_ptr` 를 통해 객체 수명과 소유권을 유연하게 관리할 수 있습니다.
- 실습을 통해 트리 구조에서 스마트 포인터를 안전하게 사용하는 패턴을 익힐 수 있습니다.

참조 사이트

- [make_shared - cppreference](#)
- [make_unique - cppreference](#)
- [Smart pointer best practices - Herb Sutter](#)

32-5 이동 의미론과 Rvalue 참조

32-5-1 이동 의미론과 Rvalue 참조

- *이동 의미론(move semantics)**은 C++11에서 도입된 기능으로, 값의 복사보다 더 효율적인 리소스 이동을 가능하게 합니다.
- **Rvalue 참조** (`T&&`)는 임시 객체(소멸 예정 객체)를 참조하며, 이동 연산자의 핵심 기반입니다.

```
std::string a = "hello";  
std::string b = std::move(a); // a의 내용을 b로 이동 (a는 비어 있음)
```

! 이동은 복사보다 효율적이며, 특히 리소스를 많이 사용하는 객체에서 성능 향상 효과가 큼니다.

32-5-2 `std::move`, `std::forward`

- `std::move` 는 Lvalue를 Rvalue로 변환합니다.
- `std::forward` 는 전달받은 인자를 원래의 값 카테고리(Lvalue or Rvalue)로 유지하며 전달합니다 (Perfect Forwarding).

```
#include <iostream>  
#include <utility>  
  
void process(std::string&& s) {  
    std::cout << "Rvalue 참조: " << s << "\n";  
}  
  
void wrapper(std::string&& s) {  
    process(std::forward<std::string>(s)); // forward로 Rvalue 유지  
}
```

32-5-3 복사 vs 이동 성능 비교

```

#include <chrono>
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<string> vec;
    constexpr int N = 100'000;
    // 실행 시간이 걸리는 코드
    // 데이터의 사이즈 크면 이동연산을 고려하자!!!
    string str = "sample text";
    for (int i = 0; i < N; ++i)
    {
        str += "sample text";
    }
    auto start = chrono::high_resolution_clock::now();
    // vec.push_back(str);
    vec.push_back(move(str));
    auto end = chrono::high_resolution_clock::now();

    cout << "이동 시간: "
         << chrono::duration<double>(end - start).count()
         << "초" << endl;

    return 0;
}

```

💡 동일 작업에서 복사보다 이동이 현저히 빠른 경우가 많음

32-5-4 실습: 이동 생성자를 가진 클래스 구현

```

#include <iostream>
#include <string>

class MyData {
    std::string data;

```

```

public:
    MyData(const std::string& d) : data(d) {
        std::cout << "복사 생성자 호출\n";
    }

    MyData(std::string&& d) noexcept : data(std::move(d)) {
        std::cout << "이동 생성자 호출\n";
    }
};

int main() {
    std::string str = "Hello";
    MyData d1(str);          // 복사 생성자
    MyData d2(std::move(str)); // 이동 생성자
    return 0;
}

```

32-5-5 핵심 요약

- 이동 의미론은 리소스를 복사 없이 이전하여 성능을 최적화하는 방법입니다.
- `std::move` 는 명시적으로 이동을 유도하고, `std::forward` 는 완벽 전달(perfect forwarding)을 구현합니다.
- 실습을 통해 이동 생성자와 복사의 차이를 체감하며 이해할 수 있습니다.

참조 사이트

- [Move semantics - cppreference](#)
- [std::move vs std::forward - Fluent C++](#)
- [C++ Move Semantics - Microsoft Learn](#)

32-6 템플릿과 STL

32-6-1 템플릿 함수와 클래스 기본

- *템플릿(Template)**은 형(type)에 독립적인 코드를 작성할 수 있게 해주는 기능입니다.

- 함수 템플릿: 다양한 형에 대해 같은 함수 로직을 적용
- 클래스 템플릿: 자료형에 따라 다른 클래스 생성

```
// 함수 템플릿 예시
template<typename T>
T add(T a, T b) {
    return a + b;
}

// 클래스 템플릿 예시
template<typename T>
class Box {
    T value;
public:
    Box(T v) : value(v) {}
    T get() { return value; }
};
```

💡 템플릿은 중복 코드를 줄이고 타입에 유연한 구조를 만들 수 있습니다.

32-6-2 `std::variant`, `std::optional`, `std::any`

- C++17에서 도입된 다양한 타입 유틸리티 템플릿입니다.

`std::variant`

- 여러 타입 중 하나만 가질 수 있는 타입 안전 유니언

```
#include <variant>
std::variant<int, std::string> data;
data = 42;
data = std::string("hello");
```

`std::optional`

- 값이 있을 수도 있고 없을 수도 있는 타입

```
#include <optional>
std::optional<int> maybeInt;
maybeInt = 10;
if (maybeInt) {
    std::cout << *maybeInt << "\n";
}
```

std::any

- 어떤 타입이든 저장 가능한 컨테이너, 타입 확인 필요

```
#include <any>
std::any anything = 123;
anything = std::string("hi");
```

⚠ any는 타입 안정성이 없고, variant와 optional이 더 권장됨

32-6-3 실습: optional 과 variant 를 활용한 안전한 데이터 처리

```
#include <iostream>
#include <variant>
#include <optional>
#include <string>

std::optional<std::string> findName(int id) {
    if (id == 1) return "Alice";
    return std::nullopt;
}

using Result = std::variant<int, std::string>;

void printResult(const Result& res) {
    std::visit([](auto&& value) {
        std::cout << "결과: " << value << "\n";
    }, res);
}
```



```
int main() {
    auto name = findName(1);
    if (name) std::cout << "이름: " << *name << "\n";

    Result r1 = 100;
    Result r2 = std::string("Success");

    printResult(r1);
    printResult(r2);
    return 0;
}
```

32-6-4 핵심 요약

- 템플릿은 타입에 독립적인 코드 작성을 가능하게 하며, 함수 및 클래스 수준에서 활용됨
- `optional`, `variant`, `any` 는 안전한 타입 처리를 지원하는 현대 C++의 주요 도구
- 실습을 통해 조건부 값 반환 및 다형적 결과 표현을 안전하게 처리하는 방법을 익힘

참조 사이트

- [Templates - cppreference](#)
- [std::variant - cppreference](#)
- [std::optional - cppreference](#)
- [std::any - cppreference](#)

32-7 멀티스레딩과 동시성

32-7-1 `std::thread`, `std::mutex`, `std::atomic`

- `std::thread` : 독립적인 실행 흐름(스레드)을 생성
- `std::mutex` : 공유 자원의 경쟁 조건(race condition)을 방지하기 위한 뮤텝스
- `std::atomic` : 원자적 연산을 통해 데이터 경합을 방지

```
#include <iostream>
#include <mutex> // mutually exclusive
#include <thread>
```

```

using namespace std;

int counter = 0;
mutex mtx;

void increment()
{
    for (int i = 0; i < 1000; ++i)
    {
        lock_guard<mutex> lock(mtx);
        ++counter;
    }
}

int main()
{
    thread t1(increment);
    thread t2(increment);
    // execution.
    // ...
    t1.join(); // t1 에서 실행되는 상황이 끝나면 대기하라!
    // ... main. 끝. t1 끝난상황..
    t2.join(); // t2 에서 실행 되는 상황이 끝나면 대기하라!
    // execution - main thread. t1 thread. t2 thread. 끝난다음
    cout << "최종 카운터 값: " << counter << endl;
}

```

💡 lock_guard는 예외 발생에도 뮤텍스를 안전하게 해제합니다.

32-7-2 `std::async` 와 `std::future`

- `std::async`: 비동기 함수 실행을 위한 표준 라이브러리 함수
- `std::future`: 비동기 작업 결과를 나중에 조회 가능

```

#include <chrono>
#include <future>
#include <iostream>
#include <vector>

```

```

using namespace std;

int slowAdd(int a, int b)
{
    for (int i = 0; i < 5; ++i)
    {
        cout << "slowAdd 실행 중 " << a << endl;
        this_thread::sleep_for(chrono::microseconds(400'000));
    }
    return a + b;
}

int main()
{
    vector<future<int>> result;
    for (int i = 0; i < 4; ++i)
        result.push_back(async(slowAdd, i + 1, 3));
    cout << "계산 중 ..." << endl;
    for (int i = 0; i < 4; ++i)
        cout << "결과 : " << result[i].get() << endl;
    return 0;
}

```

❗ .get()은 결과가 준비될 때까지 블로킹됩니다.

32-7-3 실습: 멀티스레드 기반 작업 큐 구현

```

#include <atomic>
#include <condition_variable>
#include <functional>
#include <iostream>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>

using namespace std;
using namespace chrono_literals; // 시간 코드 넣을 때

```

```

class TaskQueue
{
private:
    queue<function<void()>> tasks;
    mutex mtx;
    condition_variable cv;
    vector<thread> workers;
    atomic<bool> stop{false};

public:
    TaskQueue(size_t thread_count)
    {
        for (size_t i = 0; i < thread_count; ++i)
        {
            workers.emplace_back([this]()
            {
                while(true){
                    function<void()> task;
                    {
                        unique_lock<mutex> lock(mtx);
                        cv.wait(lock, [this]()
                            { return stop || !tasks.empty(); });
                        if (stop && tasks.empty())
                            return;
                        task = move(tasks.front());
                        tasks.pop();
                    }
                    task(); // task 실행
                }
            }
        }
    }

    void enqueue(function<void()> task)
    {
        {
            lock_guard<mutex> lock(mtx);
            tasks.push(move(task));
        }
    }
}

```

```

        cv.notify_one();
    }

    ~TaskQueue()
    {
        stop = true;
        cv.notify_all();
        for (auto &t : workers)
            t.join();
    }
};

int main()
{
    TaskQueue queue(4);
    for (int i = 0; i < 10; ++i)
    {
        queue.enqueue([i]()
            { cout << "작업 " << i << " 시작!!!" << endl; });
    }
    this_thread::sleep_for(1s);
    return 0;
}

```

32-7-4 핵심 요약

- `std::thread`, `std::mutex`, `std::atomic` 을 통해 기본적인 병렬 프로그래밍이 가능
- `std::async` 와 `std::future` 는 함수 기반의 간단한 비동기 실행 방식 제공
- 실습을 통해 안전한 작업 큐를 구성하고 스레드 간 협업 방식을 경험할 수 있음

참조 사이트

- [std::thread - cppreference](#)
- [std::mutex - cppreference](#)
- [std::async - cppreference](#)
- [C++ Concurrency in Action by Anthony Williams](#)

32-8 최종 프로젝트

32-8-1 주제: STL과 스마트 포인터를 활용한 간단한 데이터베이스 엔진

- C++ 표준 템플릿 라이브러리(STL)와 스마트 포인터(`shared_ptr` , `unique_ptr`)를 활용하여 메모리 안전하고 구조적인 데이터베이스 시스템을 구현합니다.
- 실습을 통해 템플릿, 컨테이너, 동시성, 스마트 포인터의 종합적인 이해를 평가합니다.

32-8-2 요구사항

- **데이터 삽입:** 키-값 기반의 데이터를 삽입할 수 있어야 함 (예: `insert(key, value)`)
- **데이터 검색:** 키를 이용하여 저장된 데이터를 조회 가능해야 함 (예: `find(key)`)
- **데이터 삭제:** 키를 이용하여 데이터를 삭제 가능해야 함 (예: `remove(key)`)
- **멀티스레드 지원:** 동시 요청에 대한 처리 보장 (`std::mutex` , `std::shared_mutex` , `std::thread` 활용)
- **스마트 포인터 활용:** 자원 관리를 위해 `shared_ptr` 또는 `unique_ptr` 사용 필수
- **STL 활용:** `std::unordered_map` , `std::vector` , `std::queue` 등의 컨테이너 적극 활용

32-8-3 실습: 팀별 구현 및 코드 리뷰

- 2~4명 단위로 팀을 구성하여 각 팀이 하나의 데이터베이스 엔진을 구현합니다.
- 구현 항목 예시:
 - `Database` 클래스: 삽입, 검색, 삭제 기능
 - 내부 동기화를 위한 락 구조 설계
 - 데이터 저장용 컨테이너 (`unordered_map` 등)
- 제출 전 코드 리뷰 시간을 통해 다음 항목 점검
 - 스마트 포인터 적절성
 - 동기화 정확성
 - 예외 처리와 자원 관리의 안전성

```
// 단일 키-값 DB 엔진 구조 예시 (기초)
#include <iostream>
#include <unordered_map>
```

```

#include <shared_mutex>
#include <memory>
#include <mutex>

class Database {
    std::unordered_map<int, std::string> data;
    mutable std::shared_mutex mutex;

public:
    void insert(int key, const std::string& value) {
        std::unique_lock lock(mutex);
        data[key] = value;
    }

    std::shared_ptr<std::string> find(int key) const {
        std::shared_lock lock(mutex);
        auto it = data.find(key);
        if (it != data.end()) {
            return std::make_shared<std::string>(it->second);
        }
        return nullptr;
    }

    void remove(int key) {
        std::unique_lock lock(mutex);
        data.erase(key);
    }
};

```

32-8-4 핵심 요약

- 본 프로젝트는 STL과 스마트 포인터, 멀티스레딩을 활용한 종합 실습입니다.
- 객체지향, 자원관리, 동시성 제어의 실제 적용 사례를 통해 전반적인 C++ 실무 능력을 강화합니다.
- 팀별 협업과 코드 리뷰를 통해 개발자 관점에서의 코드 품질 개선 방법을 경험합니다.

참조 사이트

- [shared_mutex - cppreference](#)

- [Smart pointers - cppreference](#)
- [unordered_map - cppreference](#)

완성 예제 코드: 멀티스레드 안전 데이터베이스 클래스

```
// 목표 : STL 과 스마트포인터를 사용해서 데이터베이스 엔진 만드세요.
// 스마트 포인터 활용 shared_ptr, unique_ptr
// 스레드 활용 multithread, mutex, atomic → 내부 동기화 위한 락 구조 설계
// 데이터 삽입 insert(key(int), value(string));
// 데이터 검색 find(key);
// 데이터 삭제 remove(key);
// class DataBase
// 저장용 컨테이너 - unordered_map 참조에 특화된!!
// 컬럼 추가(타입 정하기) - .. 도전 하실분만!
// 단일 키-값 DB 엔진 구조 예시 (기초)#include <iostream>
#include <chrono>
#include <iostream>
#include <memory>
#include <mutex>
#include <shared_mutex>
#include <string>
#include <thread>
#include <unordered_map>
#include <vector>

class ThreadSafeDB
{
private:
    std::unordered_map<int, std::shared_ptr<std::string>> db;
    mutable std::shared_mutex mutex;

public:
    void insert(int key, const std::string &value)
    {
        std::unique_lock lock(mutex);
        db[key] = std::make_shared<std::string>(value);
    }
}
```



```

std::shared_ptr<std::string> find(int key) const
{
    std::shared_lock lock(mutex);
    auto it = db.find(key);
    return (it != db.end()) ? it->second : nullptr;
}

void remove(int key)
{
    std::unique_lock lock(mutex);
    db.erase(key);
}

void printAll() const
{
    std::shared_lock lock(mutex);
    for (const auto &[key, value] : db)
    {
        std::cout << key << ": " << *value << "\n";
    }
}
};

void threadTask(ThreadSafeDB &db, int id)
{
    for (int i = 0; i < 5; ++i)
    {
        db.insert(id * 10 + i, "Data from thread " + std::to_string(id));
        std::this_thread::sleep_for(std::chrono::milliseconds(50));
    }
}

int main()
{
    ThreadSafeDB db;
    std::vector<std::thread> threads;

    for (int i = 0; i < 4; ++i)

```

```

    {
        threads.emplace_back(threadTask, std::ref(db), i);
    }

    for (auto &t : threads)
    {
        t.join();
    }

    db.printAll();
    return 0;
}

```

```

#include <iostream>
#include <memory>
#include <mutex>
#include <shared_mutex>
#include <string>
#include <thread>
#include <unordered_map>
#include <vector>

enum class ColumnType
{
    INT,
    STRING,
    FLOAT,
    BOOL
};

struct Column
{
    std::string name;
    std::string value;
};

```

```

struct Row
{
    std::vector<Column> columns;

    void addColumn(const std::string &name, const std::string &value)
    {
        columns.push_back(Column{name, value});
    }

    void print() const
    {
        for (const auto &col : columns)
        {
            std::cout << col.name << "=" << col.value << " ";
        }
        std::cout << "\n";
    }
};

class ThreadSafeDB
{
private:
    std::unordered_map<int, std::shared_ptr<Row>> db;
    std::unordered_map<std::string, ColumnType> schema;
    mutable std::shared_mutex mutex;

public:
    void registerColumn(const std::string &name, ColumnType type)
    {
        std::unique_lock lock(mutex);
        schema[name] = type;
        std::cout << "컬럼 등록됨: " << name << " (";
        switch (type)
        {
        case ColumnType::INT:
            std::cout << "INT";
            break;
        case ColumnType::STRING:

```

```

        std::cout << "STRING";
        break;
    case ColumnType::FLOAT:
        std::cout << "FLOAT";
        break;
    case ColumnType::BOOL:
        std::cout << "BOOL";
        break;
    }
    std::cout << ")\n";
}

void insert(int key, const std::vector<Column> &columns)
{
    std::unique_lock lock(mutex);
    auto row = std::make_shared<Row>();
    for (const auto &col : columns)
    {
        if (schema.find(col.name) == schema.end())
        {
            throw std::runtime_error("정의되지 않은 컬럼: " + col.name);
        }
        // 🔍 타입 검사 생략 가능, 문자열 기반
        row->addColumn(col.name, col.value);
    }
    db[key] = row;
}

void printAll() const
{
    std::shared_lock lock(mutex);
    for (const auto &[key, row] : db)
    {
        std::cout << key << ": ";
        row->print();
    }
}
};

```

```
int main()
{
    ThreadSafeDB db;

    db.registerColumn("id", ColumnType::INT);
    db.registerColumn("name", ColumnType::STRING);
    db.registerColumn("active", ColumnType::BOOL);

    db.insert(1, {"id", "101"}, {"name", "Alice"}, {"active", "true"});
    db.insert(2, {"id", "102"}, {"name", "Bob"}, {"active", "false"});

    db.printAll();

    return 0;
}
```